Multi-Tenant SaaS Application Specification

Technical Requirements, Role Matrix & Evaluation Assessment Brief (Flutter + Firebase)

Build a mini production-style SaaS application where multiple organizations use the same application while maintaining strict data isolation. The application must support real-time collaboration, role-based access control (RBAC), team management, task management, and audit logging.

1. Authentication & Organization Management

- Sign-Up & Authentication: Implement email/password sign-up and login using Firebase Authentication.
- Organization Creation: Allow a user to create a new organization and automatically become its Owner.
- Multi-Tenancy: Support users belonging to multiple organizations and switching context between them seamlessly.
- State Management: Restore authentication state reliably and gracefully handle logout, unauthorized access, and inactive/revoked memberships.

2. Roles & Permissions

Implement a fine-grained Access Control Model supporting five distinct roles: Owner, Admin, Manager, Member, and Viewer. All permission constraints must be strictly enforced at the backend level by Firebase Security Rules, not merely by hiding UI controls.

Role Action Matrix

Action	Owner	Admin	Manager	Member	Viewer
View organization data	✓	✓	✓	✓	✓
Manage organization settings	✓	✓	—	—	—
Invite / remove members	✓	✓	—	—	—
Change member roles	✓	Limited	—	—	—
Create tasks	✓	✓	✓	✓	—
Assign tasks	✓	✓	✓	—	—
Update assigned tasks	✓	✓	✓	Own	—
Delete tasks	✓	✓	✓	—	—
View audit logs	✓	✓	✓	—	—

Access Control Rules & Integrity Constraints

- **Owner Retention:** Each organization must retain at least one active Owner at all times.
- **Role Hierarchy Limits:** An Admin cannot promote themselves to Owner. Managers cannot manage organization settings or membership.
- **Task Scope Enforcement:** Members cannot modify tasks assigned to or created by another member. Viewers must strictly have read-only access.
- **Tenant Isolation:** A user must never be able to read or write another organization's private data, including through direct, unauthenticated Firestore queries.

3. Real-Time Task Management

- **Task Schema:** Create, edit, assign, and update tasks containing `title`, `description`, `status`, `priority`, `assignee`, `creator`, and `timestamps`.
- **Statuses:** `To Do`, `In Progress`, and `Done`.
- **Filtering:** Provide real-time UI filters for status, priority, and assigned team member.
- **Real-time Sync:** Data modifications made by one user must immediately reflect for all active, authorized users in the same organization without requiring manual page refresh.
- **State Robustness:** Handle loading, empty, error, and retry states. Prevent duplicate or conflicting write requests where appropriate.

4. Audit Logging

- **Tracked Events:** Record member invitations, removals, role changes, task operations (creation, updates, deletion), and organization setting updates.
- **Metadata Payload:** Each audit entry must record `actor`, `organization`, `action`, `target`, `timestamp`, and detailed change metadata.
- **Immutability & Integrity:** Ordinary users must not be permitted to edit or delete audit logs. Sensitive audit entries should be generated via trusted server-side execution where appropriate.

5. Firebase Security Requirements

- **Default-Deny Model:** Provide Firestore Security Rules starting with a global default-deny stance.
- **Tenant Isolation:** Enforce organization membership and role permissions explicitly on every `read` and `write` operation.
- **Field Validation:** Prevent users from updating their own role, jumping tenant boundaries, or tampering with protected fields (e.g., timestamps, audit logs).
- **Automated Security Tests:** Include unit/integration security rule tests, including simulated direct attack vectors against Firestore bypassing the client UI.

6. Technical & Delivery Requirements

- **Tech Stack:** Flutter and Dart with Firebase Authentication and Cloud Firestore.
- **Architecture:** Choose and justify a robust state-management solution (e.g., Provider, Riverpod, BLoC). Keep UI, data access, and business logic cleanly separated.
- **Deliverables:** Submit runnable source code, environment setup instructions, Firebase configuration requirements, and a concise architecture summary document.
- **AI Disclosure:** AI tools are permitted for coding, architecture, and debugging. You must disclose usage, verify generated code, and demonstrate full ownership of all submitted logic.

7. Live Modification & Code Review

1. **Failure Handling:** Demonstrate how the app handles network degradation (slow API), permission errors, and missing/malformed payloads gracefully.
2. **Requirement Change:** Live-implement a date-range filter that operates in tandem with existing status/priority filters.
3. **Security Review:** Walk through and demonstrate an unauthorized direct Firestore write request being rejected by security rules.
4. **Code Ownership:** Explain a non-trivial architectural choice and perform a small live modification requested by the evaluator.

Evaluation Criteria

Category	Weight	Key Assessment Focus
Functional Correctness	**25%**	Core flows work end-to-end; business constraints are enforced.
Security Rules & Tenant Isolation	**20%**	Default-deny rules, multi-tenant isolation, schema validation.
Architecture & Code Quality	**20%**	Clean separation of concerns, state management design, readability.
Real-Time Behavior & State Handling	**15%**	Active streams, offline/retry states, error/loading UI UX.
Testing & Reliability	**10%**	Automated Firestore rule tests and verification of edge cases.
AI Verification & Code Ownership	**10%**	Clarity on generated code, ability to explain and modify live code.